

Rekursion Übungen

Fibonacci

Im ersten Semester hast du Fibonacci iterativ implementiert:

```
class Fibonacci {  
  
    static long fibonacci(int n) {  
        if (n <= 1) {  
            return n;  
        }  
  
        long prev1 = 0;  
        long prev2 = 1;  
        long current = 0;  
  
        for (int i = 2; i <= n; i++) {  
            current = prev1 + prev2;  
            prev1 = prev2;  
            prev2 = current;  
        }  
  
        return current;  
    }  
}
```

Die Fibonacci-Folge ist wie folgt definiert:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ für $n \geq 2$

Schreibe die Methode so um, dass sie **rekursiv** ist.

Teste deine Implementierung mit **nachfolgenden Unit Test**.

Fibonacci: Unit Test

```
class FibonacciTest {  
  
    @Test  
    void fibonacciWith0() { assertEquals(0, Fibonacci.fibonacci(0)); }  
  
    @Test  
    void fibonacciWith1() { assertEquals(1, Fibonacci.fibonacci(1)); }  
  
    @Test  
    void fibonacciWith2() { assertEquals(1, Fibonacci.fibonacci(2)); }  
  
    @Test  
    void fibonacciWith3() { assertEquals(2, Fibonacci.fibonacci(3)); }  
  
    @Test  
    void fibonacciWith4() { assertEquals(3, Fibonacci.fibonacci(4)); }  
  
    @Test  
    void fibonacciWith5() { assertEquals(5, Fibonacci.fibonacci(5)); }  
  
    @Test  
    void fibonacciWith6() { assertEquals(8, Fibonacci.fibonacci(6)); }  
  
    @Test  
    void fibonacciWith7() { assertEquals(13, Fibonacci.fibonacci(7)); }  
  
    @Test  
    void fibonacciWith8() { assertEquals(21, Fibonacci.fibonacci(8)); }  
  
    @Test  
    void fibonacciWith9() { assertEquals(34, Fibonacci.fibonacci(9)); }  
  
    @Test  
    void fibonacciWith10() { assertEquals(55, Fibonacci.fibonacci(10)); }  
}
```

Fibonacci: Lösung

```
static long fibonacci(int n) {  
    if (n <= 1)  
        return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

SumList::sum

Erstelle eine Klasse SumList und füge folgende Methode hinzu:

```
static double sum(List<Double> numbers)
```

Die Methode gibt die **Summe aller Elemente** in numbers zurück.

Beispiel: *sum(List.of(1, 2, 3, 4, 5)); // returns 15.0*

Implementiere die Methode **rekursiv**.

Schreibe einen **Unit Test**, um deine Implementation zu testen.

SumList::sum: Lösung

Ohne zusätzlicher Kopie (mit subList):

```
final class SumList {  
  
    static double sum(List<Integer> numbers) {  
        if (numbers.isEmpty())  
            return 0;  
        return numbers.getFirst() + sum(remaining(numbers));  
    }  
  
    private static List<Integer> remaining(List<Integer> numbers) {  
        // subList returns a View on the original list (no elements are copied)  
        return numbers.subList(1, numbers.size());  
    }  
}
```

Ohne zusätzlicher Kopie (mit index):

```
static double sum(List<Integer> numbers) {  
    return sum(numbers, 0); // start with index at 0  
}  
  
static double sum(List<Integer> numbers, int index) { // helper method  
    if (index >= numbers.size())  
        return 0;  
    return numbers.get(index) + sum(numbers, index + 1);  
}
```

subList wird hier ersetzt, indem ein zusätzlicher Parameter index und eine Helper-Methode hinzugefügt wird. Der index verweist immer auf das aktuelle Element in numbers.

Mit zusätzlicher Kopie (mit Helper):

```
final class SumList {  
  
    static double sum(List<Integer> numbers) {  
        // Copies numbers into a mutable list so that immutable parameters such as List.of(1,2,3) work as well.  
        return sumInternal(new LinkedList<>(numbers));  
    }  
  
    private static double sumInternal(List<Integer> numbers) {  
        if (numbers.isEmpty())  
            return 0;  
        return numbers.removeFirst() + sumInternal(numbers);  
    }  
}
```

```
public static void main(String[] args) {  
    System.out.println(sum(List.of(1, 2, 3, 4, 5)));  
    System.out.println(sum(List.of(1, 0, -2)));  
    System.out.println(sum(List.of(0)));  
}
```

SumList::sum: Lösung Unit Test

```
class SumListTest {
```

```
    @Test
```

```
    void sumWithPositiveNumbers() {  
        var actual = SumList.sum(List.of(1, 2, 3, 4, 5));
```

```
        assertEquals(15, actual);  
    }
```

```
    @Test
```

```
    void sumWithNegativeNumbers() {  
        var actual = SumList.sum(List.of(-1, -2, -3, -4, -5));
```

```
        assertEquals(-15, actual);  
    }
```

```
    @Test
```

```
    void sumWithMixedNumbers() {  
        var actual = SumList.sum(List.of(-1, 2, 0, -3, 4, -5));
```

```
        assertEquals(-3, actual);  
    }
```

```
    @Test
```

```
    void sumWithEmptyIsZero() {  
        var actual = SumList.sum(List.of());
```

```
        assertEquals(0, actual);  
    }
```

```
    @Test
```

```
    void sumWithZeroIsZero() {  
        var actual = SumList.sum(List.of(0));
```

```
        assertEquals(0, actual);  
    }  
}
```

SumListGeneric::sum

Kopiere deine rekursive Lösung von **sum** in eine neue Klasse **SumListGeneric**.

Schreibe deine Lösung **sum** so um, dass sie eine generische Liste **beschränkt auf Numbers** als Parameter erhält (die Methode gibt weiterhin **double** zurück).

Schreibe einen **Unit Test**, um deine Implementation zu testen.

SumListGeneric::sum: Lösung

Ohne zusätzlicher Kopie (mit subList):

```
final class SumListGeneric {

    static double sum(List<? extends Number> numbers) {
        if (numbers.isEmpty())
            return 0;
        return first(numbers) + sum(remaining(numbers));
    }

    private static double first(List<? extends Number> numbers) {
        return numbers.getFirst().doubleValue(); // alternativ return (double)numbers.getFirst();
    }

    private static <T extends Number> List<T> remaining(List<T> numbers) {
        return numbers.subList(1, numbers.size()); // returns a View on the original list (no elements are copied)
    }
}
```

Ohne zusätzlicher Kopie (mit index):

```
final class SumListGeneric {

    static double sum(List<? extends Number> numbers) {
        return sum(numbers, 0); // start with index at 0
    }

    static double sum(List<? extends Number> numbers, int index) { // helper method
        if (index >= numbers.size())
            return 0;
        return numbers.get(index).doubleValue() + sumWithIndex(numbers, index + 1);
    }
}
```

Mit zusätzlicher Kopie (mit Helper):

```
final class SumListGeneric {

    static double sum(List<? extends Number> numbers) {
        // Copies numbers into a mutable list so that immutable parameters such as List.of(1,2,3) work as well.
        return sumInternal(new LinkedList<>(numbers));
    }

    private static double sumInternal(List<? extends Number> numbers) {
        if (numbers.isEmpty())
            return 0;
        return numbers.removeFirst().doubleValue() + sumInternal(numbers);
    }
}
```

```
public static void main(String[] args) {
    System.out.println(sum(List.of(1, 2, 3, 4, 5)));
    System.out.println(sum(List.of(1, 0, -2)));
    System.out.println(sum(List.of(0)));

    System.out.println(sum(List.of(1.1, 0.0, -2.2)));
}
```

FileSystemElement::elements

Lade den Code für FileSystemElement aus **Moodle** herunter.

Ergänze die Klasse **FileSystemElement** mit folgender Methode:

`abstract` Collection<FileSystemElement> `elements()`;

Implementiere die Methode für die Klasse **File** und **Directory**. Die Methode `elements` gibt eine **unmodifiable Collection** zurück, die das **Element selbst** und **alle enthaltenen Unter-Elemente** enthält.

Prüfe deine Implementierung mit folgenden Testfällen:

```
@Test
void dir1Elements() {
    var actual = dir1.elements();

    var expected = List.of(dir1, file1, dir2, dir3, file2, file3, dir4);
    assertIterableEquals(expected, actual);
}
```

```
@Test
void dir3Elements() {
    var actual = dir3.elements();

    var expected = List.of(dir3, file2, file3);
    assertIterableEquals(expected, actual);
}
```

```
@Test
void file1Elements() {
    var actual = file1.elements();

    var expected = List.of(file1);
    assertIterableEquals(expected, actual);
}
```

FileSystemElement::elements: Lösung

```
class Directory extends FileSystemElement {  
  
    // ...  
  
    @Override  
    Collection<FileSystemElement> elements() {  
        var result = new LinkedList<FileSystemElement>();  
        result.add(this);  
        for (FileSystemElement childElement : elements) {  
            result.addAll(childElement.elements());  
        }  
        return Collections.unmodifiableCollection(result);  
    }  
}
```

```
class File extends FileSystemElement {  
  
    // ...  
  
    @Override  
    Collection<FileSystemElement> elements() {  
        return List.of(this);  
    }  
}
```

MaterialManager 1/2

In der Material- und Produktionswirtschaft werden Stücklisten für die Fertigung von Produkten aus Materialien und Unterprodukten verwendet. Eine Stückliste gibt an wie viel Stück eines bestimmten Materials oder Produktes für die Fertigung eines Produktes benötigt werden.

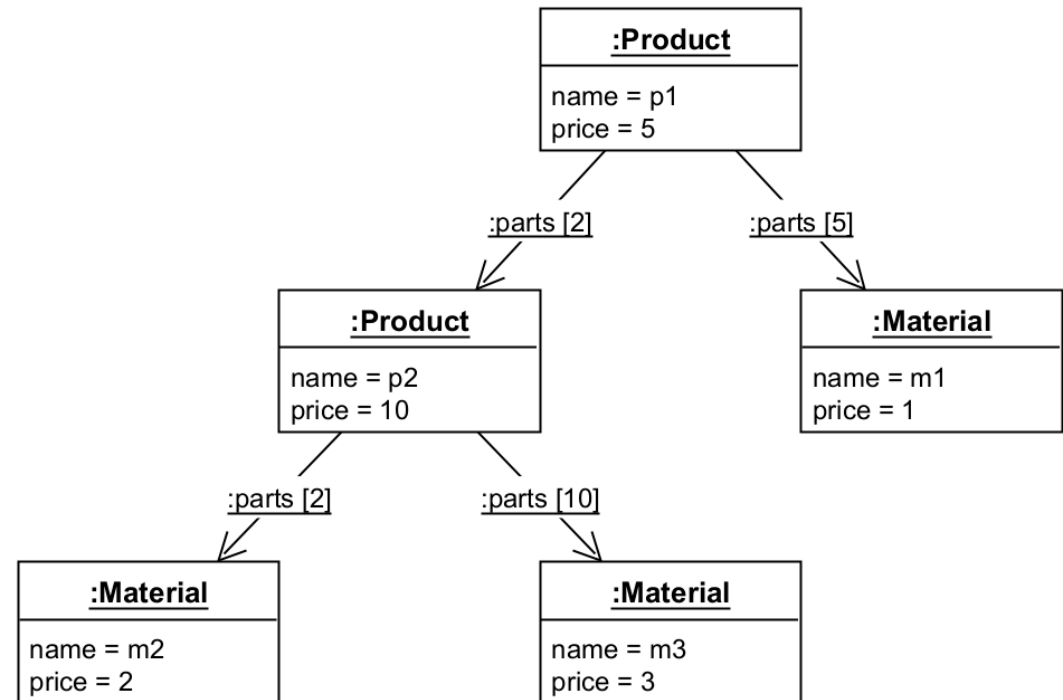
Beispiel:

Wir wollen Produkt p1 herstellen. Dafür benötigen wir 2mal Produkt p2 und 5mal Material m1.

Um p2 herzustellen, benötigen wir wiederum 2mal Material m2 sowie 10mal Material m3.

Welche Materialien muss ein Arbeiter aus dem Lager holen, um ein p1 herzustellen?

Wie hoch sind die Herstellungskosten eines p1?

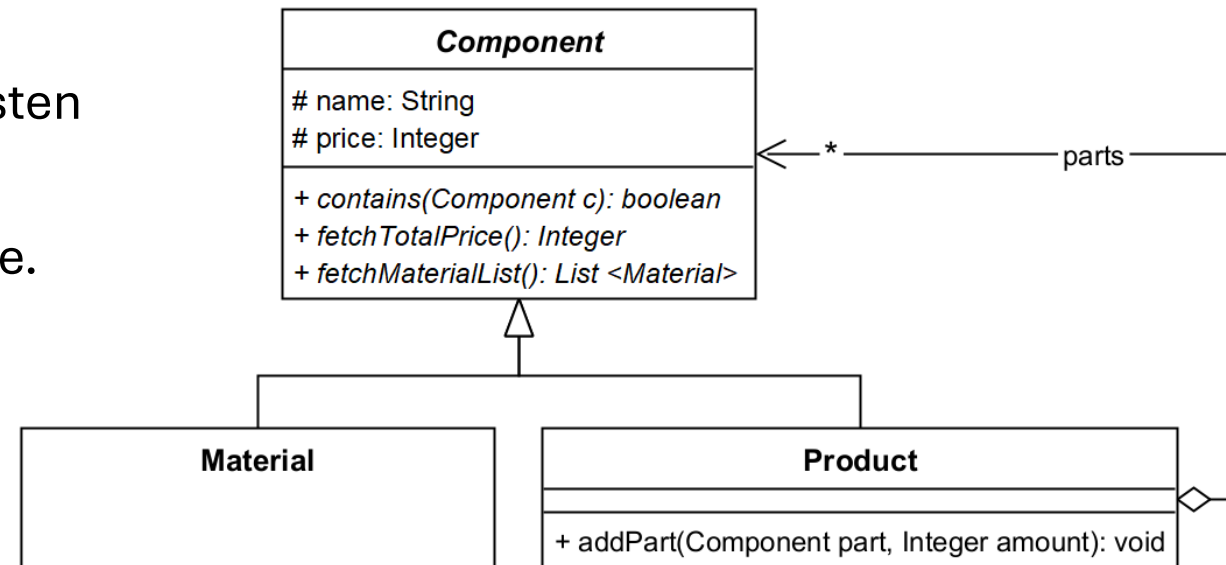


MaterialManager 2/2

1. Übersetze das UML-Klassendiagramm in Java-Code.
2. Erstelle geeignete Konstruktoren.
3. Implementiere die *contains*-Methode.
4. Implementiere die *addPart*-Methode.

Der Parameter *amount* gibt an, wie oft der *part* zum *Product* hinzugefügt werden soll.

5. Implementiere die *fetchTotalPrice*-Methode.
Sie berechnet die gesamten Herstellungskosten eines *Component*.
6. Implementiere die *fetchMaterialList*-Methode.
Sie berechnet eine Liste aller benötigten Materialien für die Herstellung eines *Component*.
7. Nutze folgenden Unit Test zum Testen.



MaterialManager: Unit Test

```
class ComponentTest {

    private Product p1;
    private Product p2;
    private Material m1;
    private Material m2;
    private Material m3;

    @BeforeEach
    void setUp() {
        p1 = new Product("p1", 5);
        p2 = new Product("p2", 10);
        p1.addPart(p2, 2);
        m1 = new Material("m1", 1);
        p1.addPart(m1, 5);
        m2 = new Material("m2", 2);
        p2.addPart(m2, 2);
        m3 = new Material("m3", 3);
        p2.addPart(m3, 10);
    }

    @Test
    void p1ContainsP2() { assertTrue(p1.contains(p2)); }

    @Test
    void p1ContainsM1() { assertTrue(p1.contains(m1)); }

    @Test
    void p1ContainsM2() { assertTrue(p1.contains(m2)); }

    @Test
    void p1ContainsM3() { assertTrue(p1.contains(m3)); }

    @Test
    void p2ContainsM2() { assertTrue(p2.contains(m2)); }

    @Test
    void p2ContainsM3() { assertTrue(p2.contains(m3)); }

    @Test
    void m1DoesNotContainM1() { assertFalse(m1.contains(m1)); }

    @Test
    void p1FetchTotalPrice() { assertEquals(98, p1.fetchTotalPrice()); }

    @Test
    void p2FetchTotalPrice() { assertEquals(44, p2.fetchTotalPrice()); }

    @Test
    void m1FetchTotalPrice() { assertEquals(1, m1.fetchTotalPrice()); }

    @Test
    void m2FetchTotalPrice() { assertEquals(2, m2.fetchTotalPrice()); }

    @Test
    void m1FetchMaterialList() { assertEquals(List.of(m1), m1.fetchMaterialList()); }

    @Test
    void p1FetchMaterialList() {
        var expected = Stream.of(
            Stream.generate(() -> m3).limit(10),
            Stream.generate(() -> m2).limit(2),
            Stream.generate(() -> m3).limit(10),
            Stream.generate(() -> m2).limit(2),
            Stream.generate(() -> m1).limit(5)
        ).flatMap(component -> component).toList();

        assertEqualsIgnoreOrder(expected, p1.fetchMaterialList());
    }

    @Test
    void p2FetchMaterialList() {
        var expected = Stream.of(
            Stream.generate(() -> m2).limit(2),
            Stream.generate(() -> m3).limit(10)
        ).flatMap(component -> component).toList();

        assertEqualsIgnoreOrder(expected, p2.fetchMaterialList());
    }

    private static <T extends Component> void assertEqualsIgnoreOrder(List<T> expected, List<T> actual) {
        assertEquals(sorted(expected), sorted(actual));
    }

    private static <T extends Component> List<T> sorted(List<T> components) {
        return components.stream()
            .sorted(Comparator.comparing(m -> m.name))
            .sorted(Comparator.comparing(m -> m.price))
            .toList();
    }
}
```

MaterialManager: Lösung

```
abstract class Component {
    protected String name;
    protected Integer price;

    public Component(String name, int price) {
        this.name = name;
        this.price = price;
    }

    public abstract boolean contains(Component component);

    public abstract Integer fetchTotalPrice();

    public abstract List<Material> fetchMaterialList();

    @Override
    public String toString() {
        return name;
    }
}
```

```
class Material extends Component {

    public Material(String name, int price) {
        super(name, price);
    }

    @Override
    public boolean contains(Component component) {
        return false;
    }

    @Override
    public Integer fetchTotalPrice() {
        return price;
    }

    @Override
    public List<Material> fetchMaterialList() {
        return List.of(this);
    }
}
```

```
class Product extends Component {

    private final List<Component> parts = new LinkedList<>();

    public Product(String name, int price) {
        super(name, price);
    }

    public void addPart(Component part, Integer amount) {
        for (int i = 0; i < amount; i++)
            parts.add(part);
    }

    @Override
    public boolean contains(Component component) {
        // Alternative zu part.equals(component) || ...
        // if (parts.contains(component)) // List::contains
        //     return true;

        for (Component part : parts) {
            if (part.equals(component) || part.contains(component)) {
                return true;
            }
        }

        return false;
    }

    @Override
    public Integer fetchTotalPrice() {
        int result = price;
        for (Component part : parts)
            result += part.fetchTotalPrice();
        return result;
    }

    @Override
    public List<Material> fetchMaterialList() {
        var result = new LinkedList<Material>();
        for (Component part : parts)
            result.addAll(part.fetchMaterialList());
        return Collections.unmodifiableList(result);
    }
}
```

MaterialManager: Lösung Map

Alternative Lösung bei der Product mit einer Map statt List implementiert wurde.
Die restlichen Klassen sind identisch mit der vorherigen Lösung.

Die Map speichert als Key die Component und als Value die amount.

```
class Product extends Component {  
  
    private final Map<Component, Integer> parts = new HashMap<>();  
  
    public Product(String name, int price) { super(name, price); }  
  
    public void addPart(Component part, Integer amount) {  
        parts.merge(part, amount, Integer::sum);  
        // Alternative statt merge:  
        // if (parts.containsKey(part)) {  
        //     var newAmount = parts.get(part) + amount;  
        //     parts.put(part, newAmount);  
        // } else {  
        //     parts.put(part, amount);  
        // }  
    }  
}
```

```
@Override  
public boolean contains(Component component) {  
    // Alternative zu part.equals(component) || ...  
    // if (parts.containsKey(component)) // Map::containsKey  
    //     return true;  
  
    for (Component part : parts.keySet()) {  
        if (part.equals(component) || part.contains(component)) {  
            return true;  
        }  
        return false;  
    }  
}  
  
@Override  
public Integer fetchTotalPrice() {  
    int result = price;  
    for (var entry : parts.entrySet()) {  
        var part = entry.getKey();  
        var amount = entry.getValue();  
        result += part.fetchTotalPrice() * amount;  
    }  
    return result;  
}  
  
@Override  
public List<Material> fetchMaterialList() {  
    var result = new LinkedList<Material>();  
    for (var entry : parts.entrySet()) {  
        var amount = entry.getValue();  
        for (int i = 0; i < amount; i++) {  
            var part = entry.getKey();  
            result.addAll(part.fetchMaterialList());  
        }  
    }  
    return Collections.unmodifiableList(result);  
}
```


Search Array

Methode `search(int[] array, int target, int index)`: Diese Methode nimmt ein Array, die zu suchende Zahl und den aktuellen Index als Parameter. Die Methode soll rekursiv durch das Array iterieren, bis sie die Zahl findet oder das Ende des Arrays erreicht.

Die Methode soll den Index der Zahl zurückgeben, wenn sie gefunden wird, oder -1, wenn die Zahl nicht im Array enthalten ist.

```
static int search(int[] array, int target, int index)
```

Search Array: Lösung

```
class SearchArray {  
  
    static int search(int[] array, int target, int index) {  
        if (index >= array.length) {  
            return -1;  
        }  
        if (array[index] == target) {  
            return index;  
        }  
        return search(array, target, index + 1);  
    }  
  
    public static void main(String[] args) {  
        int[] array = {3, 5, 7, 9, 11, 13, 15};  
        int target = 9;  
  
        int result = search(array, target, 0);  
  
        if (result != -1) {  
            System.out.println("Element " + target + " gefunden an Index " + result);  
        } else {  
            System.out.println("Element " + target + " nicht gefunden.");  
        }  
    }  
}
```